Terminologies and Column Definitions

I have separated the definitions by source below and added the observations from the raw data that affect how I use these fields later.

`pffScoutingData.csv`

There are 188,254 rows in this file.

I treat each row as one **player on one play**, or more precisely:

```
(gameId, playId, nflId)
```

The file has five main player roles, and the PFF fields are only meaningful for certain roles. That is important when building features because a NaN here is often structural rather than an ordinary missing value.

Core identifiers and player context

Column	Type	What it means
`gameId`	int	Unique ID for the game.
`playId`	int	ID for the play **within that game**. It is **not globally unique** — for example, two different games can both have a `playId` of 97. I therefore use `(gameId, playId)` together to identify a play.
`nflId`	int	Unique ID for the player.
`pff_role`	text	What the player was doing on that play.
`pff_positionLinedUp`	text	The specific position or spot where the player lined up **on that particular play**. This is different from treating the player's roster position as fixed, because the same player can line up in different spots from play to play. There are 58 distinct values in the data.

The takehome message here is I want the model to know where the player actually lined up on the play, not just what position the player is listed as on the roster.

The pressure-related PFF columns

The pressure fields need to be interpreted together with `pff_role`.

`pff_hit`, `pff_hurry`, `pff_sack`

Column	Type	What it means
`pff_hit`	0/1/NaN	Whether **this defender** recorded a hit on the play.
`pff_hurry`	0/1/NaN	Whether **this defender** recorded a hurry on the play.
`pff_sack`	0/1/NaN	Whether **this defender** recorded a sack on the play.

These fields are only meaningful for the defensive roles in this dataset — specifically `Coverage` and `Pass Rush`.

I initially expected these fields to belong exclusively to `Pass Rush`, but the raw data does not support that assumption. I discuss that below because it affects how I construct the pressure target.

`pff_beatenByDefender`

Column	Type	What it means
`pff_beatenByDefender`	0/1/NaN	Whether the pass blocker lost the matchup with the defender. This is broader than the individual hit/hurry/sack flags because a blocker can be marked as beaten even when the quarterback gets rid of the ball before the play turns into a recorded hit, hurry, or sack.

This field is only used for `Pass Block` rows.

The important distinction is that **"beaten" and "pressure event" are not necessarily the same thing**. I therefore do not automatically treat them as interchangeable labels.

`pff_hitAllowed`, `pff_hurryAllowed`, `pff_sackAllowed`

Column	Type	What it means
`pff_hitAllowed`	0/1/NaN	Whether this blocker's assignment resulted in a hit being allowed.
`pff_hurryAllowed`	0/1/NaN	Whether this blocker's assignment resulted in a hurry being allowed.
`pff_sackAllowed`	0/1/NaN	Whether this blocker's assignment resulted in a sack being allowed.

These are only meaningful for `Pass Block` rows.

For the pressure-allowed problem, I combine these fields into a single target:

```
pressure_allowed = 1
if any of hit / hurry / sack was allowed
```

That gives me a single binary outcome while keeping the original PFF fields available for checking and analysis.

`pff_nflIdBlockedPlayer`

Column	Type	What it means
`pff_nflIdBlockedPlayer`	int/NaN	The `nflId` of the specific rusher this blocker was primarily matched against.

This is only populated for `Pass Block` rows.

I use this field as the explicit blocker-rusher relationship when it is appropriate, but I do **not** assume it solves every assignment situation. In particular, it cannot by itself represent double teams or unblocked rushers, which is why the rusher-side problem uses nearest-blocker distance from the tracking data instead.

`pff_blockType`

Column	Type	What it means
`pff_blockType`	text	PFF's short code for the blocking technique used.

This is only meaningful for `Pass Block` rows.

For the block-type model, the distribution is heavily skewed. PP is very common, while some techniques have fewer than 150 examples. I therefore collapse techniques with fewer than 150 observations into `OTHER` before modeling.

`pff_backFieldBlock`

Column	Type	What it means
`pff_backFieldBlock`	0/1/NaN	Whether the blocker started the play in the backfield rather than on the offensive line. A typical example would be a running back staying in to block.

Again, this is only meaningful for `Pass Block` rows.

`pff_role`

This is the field I use to distinguish the main player roles in the dataset.

Value	Count	What it means
`Coverage`	57,765	Defender whose primary role on the play was covering a receiver/route.
`Pass Block`	46,057	Offensive player whose role was pass blocking.
`Pass Route`	39,513	Offensive player — such as a WR, TE, or RB — running a route.
`Pass Rush`	36,362	Defender whose role was rushing the passer.
`Pass`	8,557	The quarterback's own row — the passer.

These role counts add up to the full 188,254 rows.

The role field is also important for interpreting the rest of the dataset because many NaNs are not random missing observations. They are a result of the fact that a particular field simply does not apply to that player's role on that play.

One thing I did not want to assume about pressure stats

`pff_hit` / `pff_hurry` / `pff_sack` are not exclusive to `Pass Rush`

I expected these fields to be populated only for `Pass Rush` rows.

but if I found that they are non-null for both:

- `Coverage` — 57,765 rows
- `Pass Rush` — 36,362 rows

And there are even a small number of `Coverage`-labeled players who record a positive pressure event:

- `pff_sack == 1` for 17 `Coverage` rows
- `pff_hit == 1` for 20 `Coverage` rows

So I am not treating `pff_role == "Pass Rush"` as a hard filter for pressure events.

The practical interpretation is that a defender whose primary job was coverage can occasionally still be responsible for a hit or sack. That can happen in situations such as a delayed or disguised blitz, or during a scramble drill.

This is one of those places where the raw data is more useful than the assumption I would have made from the column names alone.

`team_information.csv`

This file is much simpler: 32 rows, one per NFL team.

Column	Meaning
`team_abbr`	Two- or three-letter team code, for example `ARI` or `KC`. This matches the `team` column in the tracking data.
`team_nick`	Team nickname, for example `Cardinals`.
`team_color`	Team color stored as a hex color code.
`team_color2`	Secondary team color stored as a hex color code.

The main reason I care about this file is the `team_abbr` mapping: it gives me a clean way to connect the team identifiers used in the tracking data to human-readable team information.

`tracking_parquets/week*.parquet`

The weekly tracking files contain roughly 1.1 million rows per week.

The unit of observation is different from the PFF file:

```
one row = one player (or the football) at one frame
```

So this is fundamentally **frame-level tracking data**, rather than one-row-per-player-per-play data.

That distinction becomes important when I engineer features such as displacement, orientation variability, closing velocity, and nearest-player distance.

Tracking identifiers

Column	Type	What it means
`gameId`	int	Same game identifier used in `pffScoutingData.csv`.
`playId`	int	Same play identifier used in `pffScoutingData.csv`. It is only unique within `gameId`, so `(gameId, playId)` remains the play key.
`nflId`	int	Same player identifier used in the PFF data.

There is one special case here: **the football itself also gets a tracking row on every frame**.

For that row:

```
nflId = NaN
team = "football"
```

So not every tracking row represents a player.

Frame timing

Column	Type	What it means
`frameId`	int	Frame number within the play, starting at 1. Frames are 0.1 seconds apart, giving 10 samples per second (10 Hz).
`time`	timestamp	Wall-clock time associated with that frame.

Because the sampling interval is 0.1 seconds, consecutive frames give me a regular 10 Hz sequence that I can use for movement-based feature engineering.

Player and team information

Column	Type	What it means
`jerseyNumber`	int	The player's jersey number.
`team`	text	Team abbreviation, or the literal string `"football"` for the football's own tracking row.
`playDirection`	text	`"left"` or `"right"` — the direction in which the offense is moving. I use this when I need to interpret `x` consistently relative to the direction of the play.

The `playDirection` field is particularly useful because the absolute field coordinates are not the same as the player's football-relative direction. If I want a movement feature such as forward displacement, I need to account for whether the offense is moving left or right.

Movement and field position

Column	Type	What it means
`x`	float (yards)	Player position along the length of the field.
`y`	float (yards)	Player position across the width of the field.
`s`	float (yards/sec)	Instantaneous speed.
`a`	float (yards/sec²)	Instantaneous acceleration.
`dis`	float (yards)	Distance traveled since the previous frame.
`o`	float (degrees, 0–360)	**Orientation** — which way the player's body is facing.
`dir`	float (degrees, 0–360)	**Direction** — which way the player is actually moving.

The distinction between `o` and `dir` is important.

A player can be **facing one direction while moving in another direction**, so I do not treat orientation and movement direction as interchangeable measurements.

That becomes particularly relevant for the role and block-technique problems, where orientation variability and direction variability are part of the feature set.

`event`

Column	Type	What it means
`event`	text/NaN	A label attached to the relatively small number of frames where something notable happened, such as `ball_snap`, `pass_forward`, or `qb_sack`. It is `NaN` on all other frames.

Most tracking frames therefore do **not** have an event label.

I treat these as occasional event markers rather than a continuously populated feature.

Observed ranges in `week1.parquet`

As a sanity check, these are the ranges I observed in the first week's tracking data.

`x`

Observed range:

```
0.25 to 119.72 yards
```

The field is 120 yards long when both end zones are included.

`y`

Observed range:

```
−2.61 to 57.01 yards
```

The field itself is approximately 53.3 yards wide.

Values outside that range are not necessarily errors. They can happen when a player runs out of bounds.

`s`

Observed maximum:

```
28.3 yd/s
```

`a`

Observed maximum:

```
50.7 yd/s²
```